

Bounded Buffer – Consumatore

```
item next_consumed;

while (true) {
    while (isEmpty)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    size--;

    /* consume the item in next consumed */
}
```

Interprocess Communication – Message Passing

- Meccanismi che i processi usano per comunicare e per sincronizzarsi
- Messaggi – processi comunicano senza ricorrere a variabili condivise, ma si scambiano messaggi
 - La dimensione del message può essere fissa o variabile
- IPC fornisce solitamente due operazioni:
 - `send(message)`
 - `receive(message)`

Message Passing

- Se i processi P e Q vogliono comunicare devono:
 - Stabilire un canale (**communication link**) tra loro
 - Scambiare messaggi via send/receive

- Implementazione:
 - Come stabilire il canale?
 - Può il canale coinvolgere più di due processi?
 - Quanti canali di comunicazione tra coppie di processi comunicanti?
 - Capacità di un canale?
 - La dimensione del messaggio scambiato sul canale è fisso o variable?
 - Il canale è unidirezionale o bi-direzionale?

Message Passing

- Implementazione di un canale di comunicazione:
 - Non consideriamo tanto il canale fisico ...
 - ▶ Memoria condivisa
 - ▶ Bus hardware
 - ▶ Rete
 - ... quanto il canale logico:
 - ▶ Comunicazione diretta o indiretta
 - ▶ Comunicazione sincrona o asincrona
 - ▶ Buffering dei messaggi automatico o esplicito

Comunicazione Diretta

- Processi devono nominarsi esplicitamente (**naming**):
 - `send(P, message)` – invia message al processo P
 - `receive(Q, message)` – riceve message dal processo Q
- Propertà del canale di comunicazione:
 - Il canale è stabilito automaticamente tra coppie di processi comunicanti (che conoscono l'identità reciproca)
 - Il canale è stabilito tra esattamente due processi
 - Per ogni coppia c'è esattamente un canale
 - Il canale può essere unidirezionale o bi-direzionale
- Variante - comunicazione **asimmetrica**:
 - `send(P, message)` – invia message al processo P
 - `receive(id, message)` – riceve message da qualunque processo (id ricevuto in ingresso)
- Problema: modularità, modificando l'identità di un processo richiede revisione delle comunicazione

Comunicazione Indiretta

- Messaggi mandati/ricevuti su **mailboxes** (o **porte**):
 - Ogni mailbox ha un unico id
 - Processi possono comunicare solo se condividono una mailbox

- Propertà del canale:
 - Canale stabilito solo se i processi condividono un mailbox
 - Canale può essere associato a più processi
 - Coppie di processi possono condividere più canali
 - Canali sia unidirezionali che bi-direzionali

Comunicazione Indiretta

□ Operazioni

- Creare una nuova mailbox (porta)
- Inviare/ricevere messaggi (send/receive) via mailbox
- distruggere una mailbox

□ Primitive:

send(*A, message*) – invia *message* alla mailbox *A*

receive(*A, message*) – ricevi *message* dalla mailbox *A*

□ Propertà del canale:

- Canale stabilito solo se i processi condividono un mailbox
- Canale può essere associato a più processi
- Coppie di processi possono condividere più canali
- Canali sia unidirezionali che bi-direzionali

Comunicazione Indiretta

- Condivisione del mailbox
 - Si supponga che P_1 , P_2 , e P_3 condividano la mailbox A
 - Se P_1 , invia M ad A e P_2 e P_3 ricevono da A
 - Chi prende il messaggio M?

- Soluzione dipendono dal metodo scelto:
 - Permettere una connessione tra solo due processi alla volta
 - Permettere ad un processo alla volta di eseguire *receive()*
 - Lasciare al sistema decidere il ricevente (al mittente verrà notificata la decisione)

- La mailbox può essere proprietà del SO o di un processo (nello spazio di indirizzamento del processo).
 - Nel caso di mailbox di proprietà di un processo utente, questo può solo ricevere, gli altri possono solo inviare (alla terminazione del processo è dismessa)
 - Se la mailbox è del SO questa non viene dismessa

Sincronizzazione

- ❓ La comunicazione tra processi avviene con chiamate *send-receive*
- ❓ Il message passing può essere *bloccante* o *non bloccante*
- ❓ **Bloccante** considerato **sincrono**
 - ❓ **Blocking send** -- mittente è bloccato finché il messaggio non viene ricevuto
 - ❓ **Blocking receive** -- destinatario bloccato finché il messaggio non è disponibile
- ❓ **Non bloccante** considerato **asincrono**
 - ❓ **Non-blocking send** -- mittente invia il messaggio e continua ad eseguire
 - ❓ **Non-blocking receive** -- destinatario riceve:
 - ❓ messaggio valido, oppure
 - ❓ messaggio nullo

Diverse combinazioni di politiche

- ❓ Se bloccati sia il mittente che il destinatario abbiamo un **rendezvous**

Sincronizzazione

- ❑ Con sincronizzazione bloccante il problema del produttore consumatore diventa triviale:
 - ❑ Il produttore invia e attende il consume
 - ❑ Il consumatore attende e consuma

```
message next_produced;  
while (true) {  
    /* produce an item in next produced */  
    send(next_produced);  
}
```

```
message next_consumed;  
while (true) {  
    receive(next_consumed);  
  
    /* consume the item in next consumed */  
}
```

Buffering

- Lo scambio di messaggi utilizza un canale di comunicazione basato su code temporanee
- Le code sono implementate in tre modi:
 - **Capacità zero** – zero messaggi in coda, il mittente aspetta che il destinatario riceva (rendezvous)
 - **Capacità limitata** – capacità di n messaggi in coda, il mittente aspetta se il canale è pieno
 - **Capacità illimitata** – la coda non ha lunghezza predefinita, il mittente non aspetta mai
- Capacità zero è detto **senza buffering**
- Capacità non zero è detto con **buffering automatico**

Esempio di IPC Systems - POSIX

? POSIX Shared Memory

? Un processo crea un segmento di shared memory con la call
`shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`

- ? nome, modalità di apertura/creazione, permessi (read e write)
- ? restituisce un intero (descrittore di file)
- ? anche usata per aprire un segmento esistente per dividerlo

? Creato l'oggetto si setta la dimensione

```
ftruncate(shm_fd, 4096);
```

- ? Setta la dimensione 4096 byte

? Crea un memory-mapped file

```
mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

- ? `MAP_SHARED` specifica che i cambiamenti sono condivisi

? È quindi possibile scrivere su shared memory (puntatore) come su file

```
sprintf(shared memory, "Writing to shared memory");
```

IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```

Aperta con nome "OS" in modalità R/W - se non esiste creata con permessi R/W -

Dimensione SIZE

Mappata in memoria in modalità MAP_SHARED - cambiamenti visibili a chi condivide la memoria

PROT_WRITE permette la scrittura

Puntatore ptr incrementato in modalità non automatica

IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

Aperta con nome "OS" in modalità lettura con permessi R/W

Dimensione SIZE

Mappata in memoria in modalità MAP_SHARED - cambiamenti visibili a chi condivide la memoria

PROT_READ permette la lettura

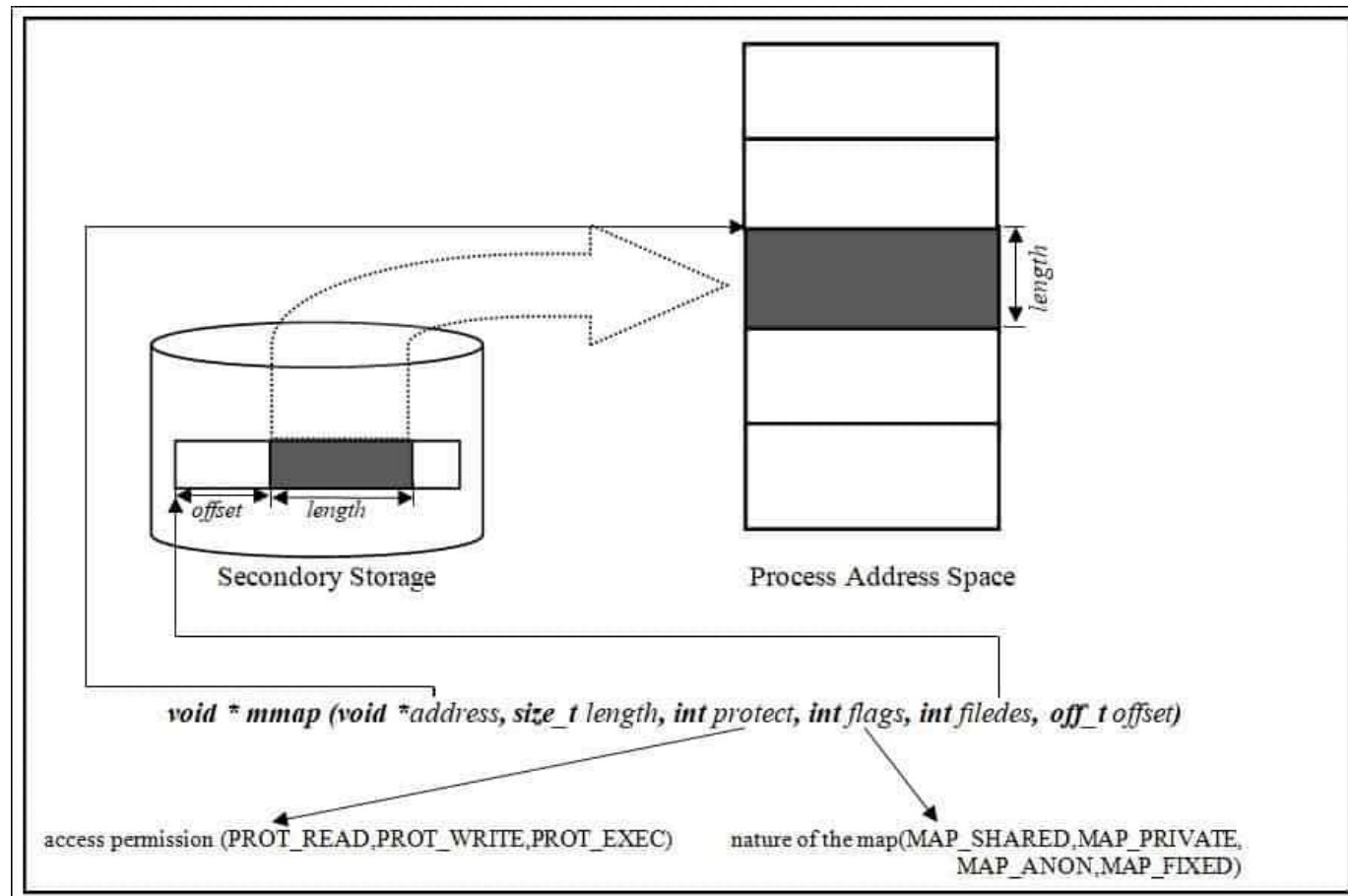
Rimuove l'oggetto shared mem

POSIX mmap

- La funzione **mmap**: mappa in process address space un file o un device
 - Una volta mappato accessibile in modo diretto

`void * mmap (void *address, size_t length, int protect, int flags, int filedes, off_t offset)`

- **address**: indirizzo di partenza
- **length**: bytes richiesti
- **protect**:
PROT_READ | PROT_WRITE |
PROT_EXEC | PROT_NONE
- **flags**:
MAP_SHARED, MAP_PRIVATE
MAP_ANON, MAP_FIXED
- **fildes**: descrittore di file
- **offset**: offset del file

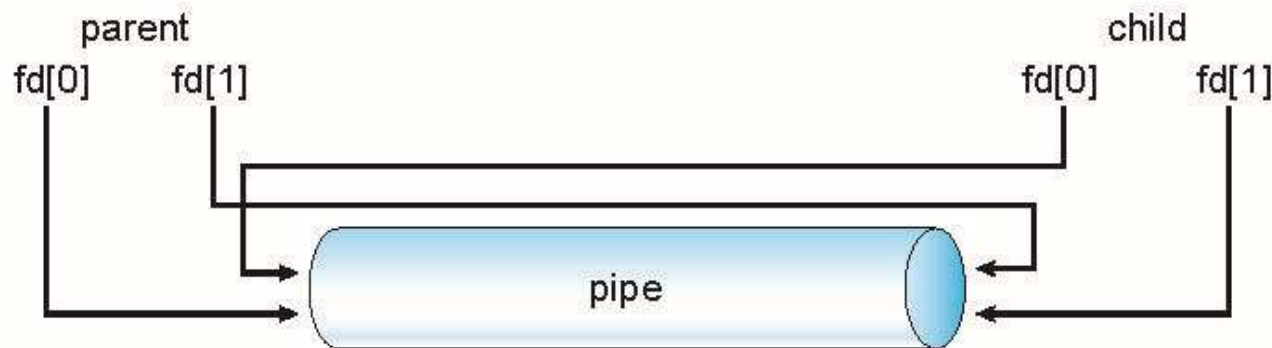


Pipe

- ❑ Flusso di dati che permette a due processi di comunicare
- ❑ Uno dei primi meccanismi di comunicazione in UNIX
- ❑ Problematiche:
 - ❑ Comunicazione unidirezionale o bidirezionale?
 - ❑ Se two-way communication, è half o full-duplex?
 - ❑ Ci deve essere una relazione (i.e., **parent-child**) tra i processi comunicanti?
 - ❑ Si possono usare le pipes in una rete?
- ❑ **Pipe ordinarie**
 - ❑ Non può essere usata da processi che non hanno creato
 - ❑ Processo padre crea la pipe a la usa per comunicare con un processo figlio
- ❑ **Named pipes**
 - ❑ Possono essere usate senza una relazione parentale tra processi

Pipe Ordinarie

- ? Pipe ordinarie comunicano in modalità produttore-consumatore
- ? Produttore scrive su un'estremità (**write-end** della pipe)
- ? Consumatore legge dall'altra estremità (**read-end** della pipe)
- ? Le pipe ordinarie sono unidirezionali
- ? Richiedono una relazione parentale tra i processi che comunicano



- ? Windows le chiama **anonymous pipes**

Pipe Ordinarie

- ❓ La funzione pipe in Unix

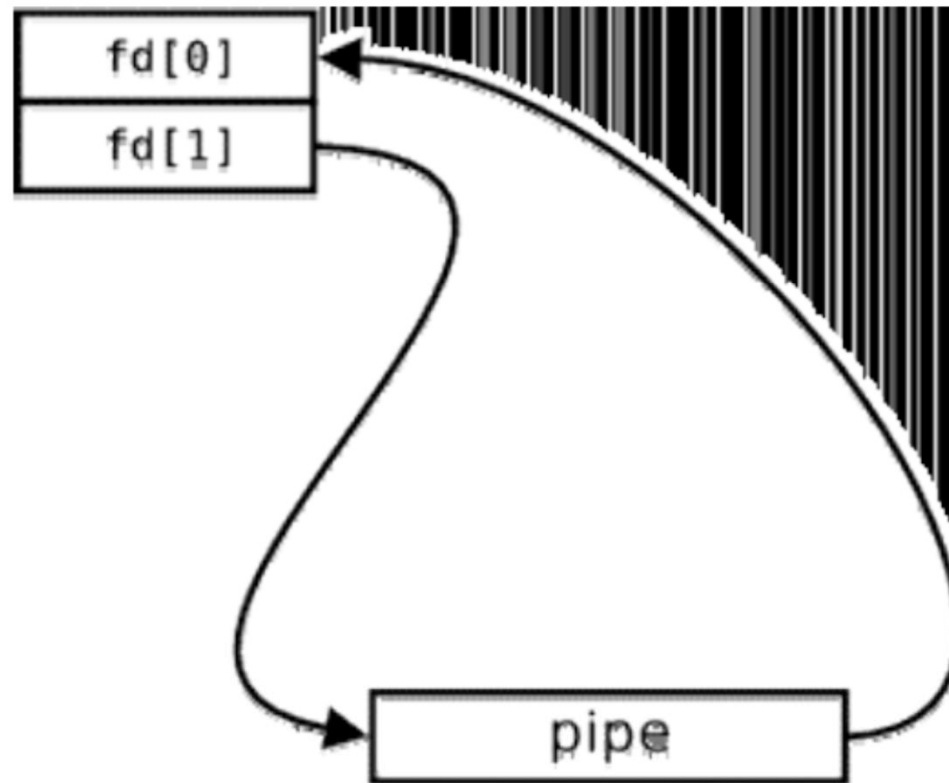
```
#include <unistd.h>

int pipe ( int filedes[2] );
```

- l'argomento *filedes* è costituito da due descrittori di file:
 - *filedes[0]* è aperto in lettura e rappresenta il lato in lettura della pipe ;
 - *filedes[1]* è aperto in scrittura e rappresenta il lato in scrittura della pipe;
 - l'output di *filedes[1]* è l'input per *filedes[0]*;
- restituisce 0 in caso di successo, -1 altrimenti.

Pipe Ordinarie

- ❓ La funzione pipe in Unix

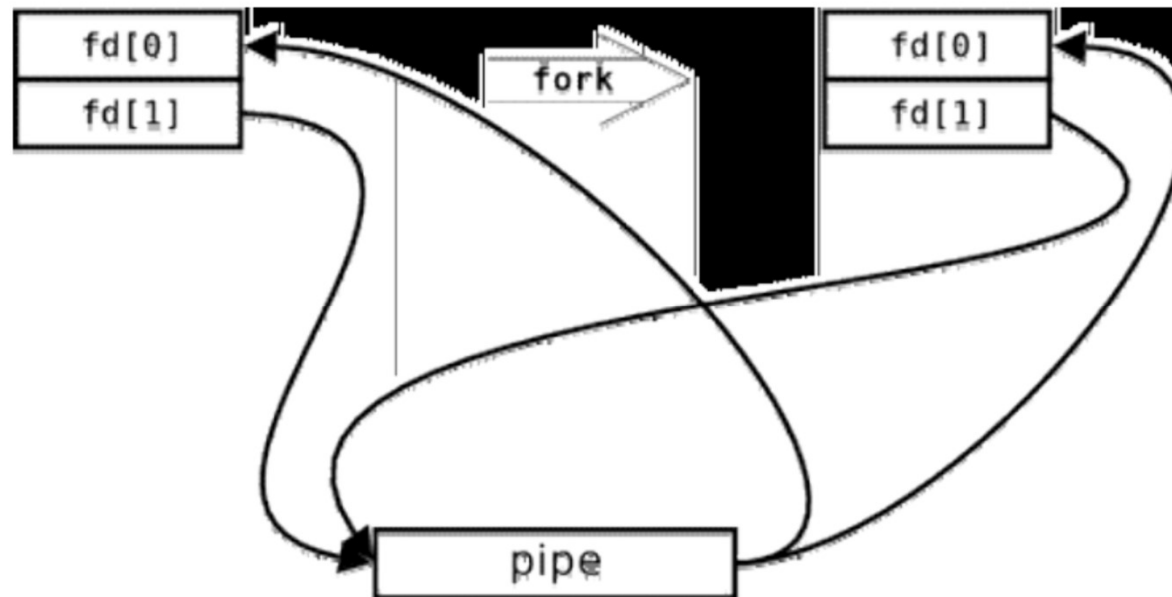


Generata da un singolo processo ha poca utilità ...

Pipe Ordinarie

❓ La funzione pipe in Unix

- Tipicamente, un processo crea una pipe e poi chiama fork



Pipe Ordinarie

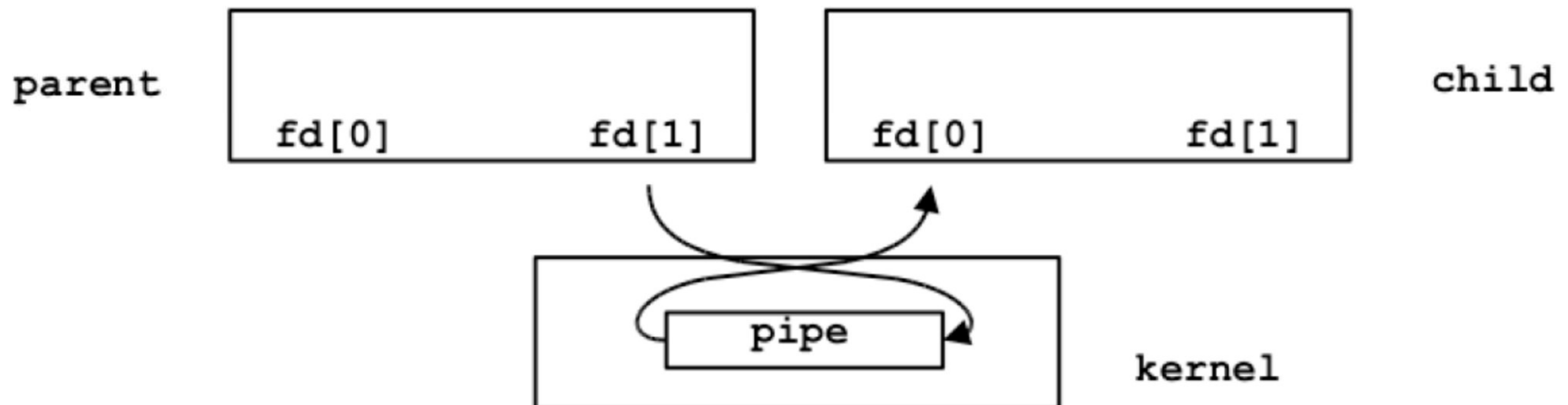
❓ La funzione pipe in Unix

- **Come utilizzare i pipe?**

- Cosa succede dopo la `fork` dipende dalla direzione dei dati
- I canali non utilizzati vanno chiusi

- **Esempio: parent → child**

- Il parent chiude l'estremo di read (`close(fd[0]);`)
- Il child chiude l'estremo di write (`close(fd[1]);`)



Pipe Ordinarie

- ❓ La funzione pipe in Unix

```
int fd[2];

if (pipe(fd) < 0)
    perror("pipe"), exit(1);
if ( (pid=fork()) < 0 )
    perror("fork"), exit(1);
else if (pid>0) { // padre
    close(fd[0]);
    write(fd[1], "ciao!", 5);
} else { // figlio
    close(fd[1]);
    n = read(fd[0], buf, sizeof(buf));
    write(STDOUT_FILENO, buf, n);
}
```

Pipe Ordinarie

❓ La funzione pipe in Unix

```
/* pipe1: invio di dati da un genitore ad un figlio */

#include <stdio.h>
#include <unistd.h>
#define MAXLINE 64

int main(void)
{
    int n, fd[2];
    pid_t pid;
    char line[MAXLINE];

    if (pipe(fd) < 0) perror("pipe"), exit(1);

    if ( (pid = fork()) < 0) perror("fork"), exit(1);

    else if (pid > 0) { /* genitore */
        close(fd[0]);
        write(fd[1], "hello world\n", 12);
    }
    else { /* figlio */
        close(fd[1]);
        n = read(fd[0], line, MAXLINE);
        write(STDOUT_FILENO, line, n);
    }
    exit(0);
}
```

Pipe Ordinarie

❓ La funzione pipe in Unix

- All'inizio una pipe è vuota
- write aggiunge dati alla pipe
- read legge e *rimuove* dati dalla pipe
 - non si possono leggere piu' volte gli stessi dati da una pipe
 - non si puo' chiamare lseek su una pipe
 - i dati si ottengono in ordine First In First Out
- una pipe con una estremita' chiusa si dice rotta (broken)

Pipe Ordinarie

❓ La funzione pipe in Unix

- Scrivere: `write` aggiunge i suoi dati alla pipe
 - se la pipe e' rotta, viene generato il segnale SIGPIPE e `write` restituisce un errore
- Leggere: `read(fd[0], buf, 100)`
 - meno di 100 bytes nella pipe: `read` legge l'intero contenuto della pipe
 - piu' di 100 bytes nella pipe: `read` legge i primi 100 bytes
 - pipe vuota: `read` si blocca in attesa di dati
 - pipe vuota e rotta: `read` restituisce 0

Pipe Ordinarie

❓ La funzione pipe in Windows (pipe anonime)

❓ Come in Unix, processi imparentati

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
```

```
#define BUFFER_SIZE 25
```

```
int main(VOID)
{
```

```
    HANDLE ReadHandle, WriteHandle;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    char message[BUFFER_SIZE] = "Greetings";
    DWORD written;
```

```
    /* set up security attributes allowing pipes to be inherited */
    SECURITY_ATTRIBUTES sa = {sizeof(SECURITY_ATTRIBUTES),NULL,TRUE};
    /* allocate memory */
    ZeroMemory(&pi, sizeof(pi));
```

struttura per far ereditare gli
handle al processo figlio

va allocate memoria per la pipe

```
    /* create the pipe */
    if (!CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
        fprintf(stderr, "Create Pipe Failed");
        return 1;
    }
```

Pipe Ordinarie

? La funzione pipe in Windows (pipe anonime)

? Come in Unix, processi imparentati

```
/* establish the START_INFO structure for the child process */
GetStartupInfo(&si);
si.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);

/* redirect standard input to the read end of the pipe */
si.hStdInput = ReadHandle;
si.dwFlags = STARTF_USESTDHANDLES;

/* don't allow the child to inherit the write end of pipe */
SetHandleInformation(WriteHandle, HANDLE_FLAG_INHERIT, 0);

/* create the child process */
CreateProcess(NULL, "child.exe", NULL, NULL,
    TRUE, /* inherit handles */
    0, NULL, NULL, &si, &pi);

/* close the unused end of the pipe */
CloseHandle(ReadHandle);

/* the parent writes to the pipe */
if (!WriteFile(WriteHandle, message, BUFFER_SIZE, &written, NULL))
    fprintf(stderr, "Error writing to pipe.");

/* close the write end of the pipe */
CloseHandle(WriteHandle);

/* wait for the child to exit */
WaitForSingleObject(pi.hProcess, INFINITE);
CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
return 0;
}
```

Prepara le strutture del figlio
lo stdout e lo stdin

Ridirezione dello standard input per
leggere da pipe

Flag del figlio settato a TRUE per far
ereditare gli handle

Pipe Ordinarie

? La funzione pipe in Windows (pipe anonime)

? Come in Unix, processi imparentati

```
#include <stdio.h>
#include <windows.h>

#define BUFFER_SIZE 25

int main(VOID)
{
    HANDLE Readhandle;
    CHAR buffer[BUFFER_SIZE];
    DWORD read;

    /* get the read handle of the pipe */
    ReadHandle = GetStdHandle(STD_INPUT_HANDLE); Legge da stdin, ma è
                                                    ridirezionato su pipe

    /* the child reads from the pipe */
    if (ReadFile(ReadHandle, buffer, BUFFER_SIZE, &read, NULL))
        printf("child read %s",buffer);           Scrive su stdout
    else
        fprintf(stderr, "Error reading from pipe");

    return 0;
}
```

Pipe con Nome

- ❑ Named Pipes sono più potenti delle pipe ordinarie
- ❑ La comunicazione è bidirezionale
- ❑ Non è necessaria una relazione parentale tra processi comunicanti
- ❑ Molti processi possono usare le named pipe per comunicare
- ❑ Sia su UNIX che su Windows

Pipe con Nome in Unix

I file speciali **FIFO** (**pipe con nome**) consentono di superare alcune delle limitazioni delle pipe. Essi infatti, rispetto a queste ultime, offrono i seguenti vantaggi:

- una volta creati, esistono nel file system fin tanto che non vengono esplicitamente cancellati;
- possono essere usati da processi che **non hanno un comune antenato**.

I file FIFO possono essere **creati** in due modi:

- attraverso la shell, con il **comando mkfifo**;
- all'interno di un programma, con la chiamata alla **funzione mkfifo**.

Una volta creato un file FIFO, su di esso si possono effettuare le **operazioni usuali di IO** su file (**open, read, write, close, ...**)

Pipe con Nome in Unix

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define MAX_BUF_SIZE 1000

int main(int argc, char *argv[]){

int fd, ret_val, count, numread;
char buf[MAX_BUF_SIZE];

/* Create the named - pipe */
ret_val = mkfifo("miafifo", 0666);
if ((ret_val == -1) && (errno != EEXIST)) {
    perror("Error creating the named pipe");
    exit (1); }

/* Open the pipe for reading */
fd = open("miafifo", O_RDONLY);

/* Read from the pipe */
numread = read(fd, buf, MAX_BUF_SIZE);
buf[numread] = '\0';
printf("Server : Read From the pipe : %s\n", buf);

/* Convert to the string to upper case */
count = 0;
while (count < numread) {
    buf[count] = toupper(buf[count]);
    count++;
}
printf("Server : Converted String : %s\n", buf);
}
```

Codice Server

Pipe con Nome in Unix

```
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

Codice Client

```
int main(int argc, char *argv[])
{
    int fd;

    /* Check if an argument was specified. */

    if (argc != 2) {
        printf("Usage : %s <string to be sent to the server>\n", argv[0]);
        exit (1);
    }

    /* Open the pipe for writing */
    fd = open("miafifo", O_WRONLY);

    /* Write to the pipe */
    write(fd, argv[1], strlen(argv[1]));
}
```

Esempio esecuzione:

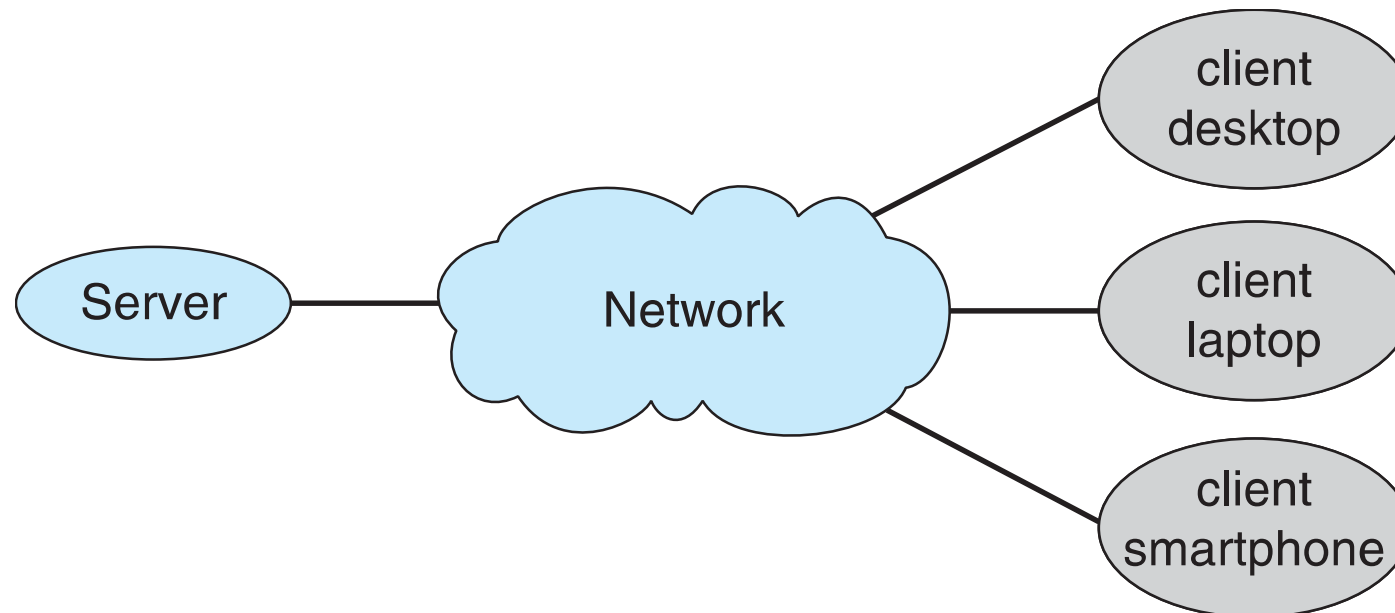
- `./servFifo &`
- `./clientFifo prova`
- Server : Read From the pipe : prova
- Server : Converted String : PROVA

Comunicazione in Sistemi Client-Server

- Per la comunicazione client-server vediamo due metodi
 - Sockets
 - Remote Procedure Call

Sistemi Client-Server

- Sistemi Client-Server
 - Tipica architettura di rete dove Sistemi **Server** rispondono alle richieste dei Sistemi **client**



Sistemi Distribuiti

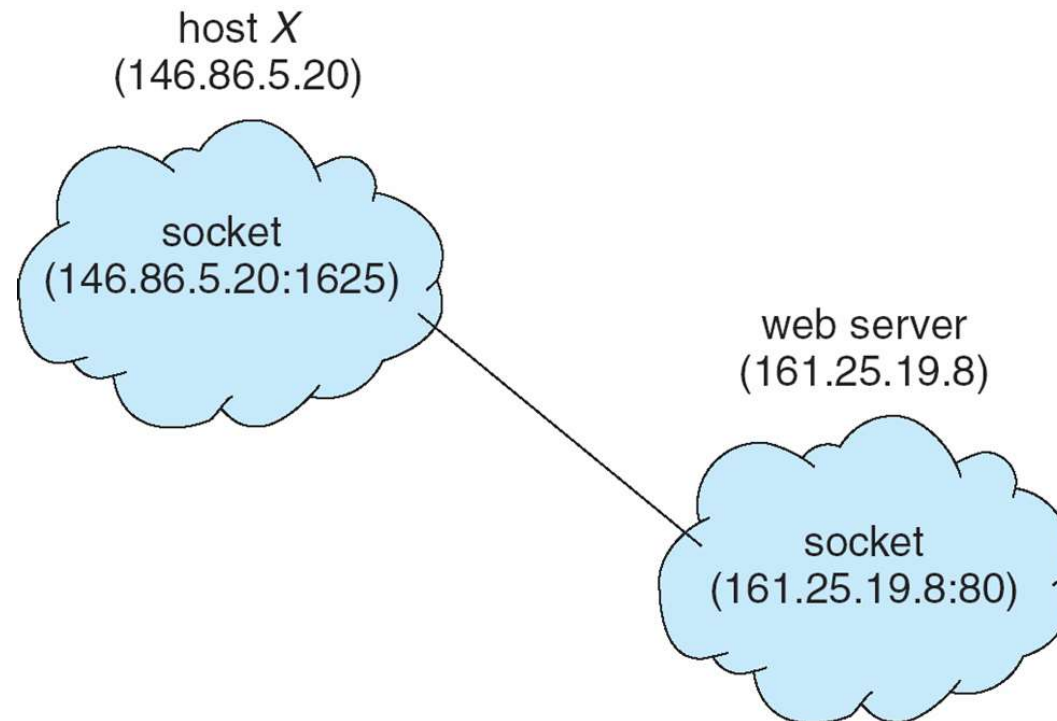
- Sistemi distribuiti
 - Insiemi di calcolatori fisicamente separati con caratteristiche eterogenee connessi in una rete per consentire agli utenti l'accesso alle risorse distribuite
 - La rete fornisce un canale di comunicazione (es., tcp/ip)
 - Differenti tipi di reti
 - **Local Area Network (LAN)**
 - **Wide Area Network (WAN)**
 - **Metropolitan Area Network (MAN)**
 - **Personal Area Network (PAN)**

Socket

- Una **socket** è un punto finale (endpoint) di comunicazione
- Introdotti nel 1983 in Berkeley Software Distribution (BSD)
 - Berkeley socket API
- La comunicazione avviene tra coppie di socket
 - Su stessa macchina (IPC locali) o diverse (IPC di rete)
- Una socket internet è identificata da un indirizzo e una porta:
161.25.19.8:1625 indica la porta **1625** sull'host **161.25.19.8**
- La **porta** è un numero che differenzia diversi servizi su un host
- Tutte le porte sotto 1024 sono usate per servizi standard
 - Es. FTP 21, SSH 22, 25 smtp, 80 HTTP, etc. (vedi www.iana.org)
- Un IP address speciale 127.0.0.1 (**loopback**) si riferisce al sistema in cui il processo gira

Comunicazione Socket

- Comunicazione Client-Server
 - Server in ascolto su una porta e Client richiede servizio
 - Processo Client stabilisce una comunicazione tramite una porta
 - ▶ Connessione unica per processo (altro Client altra porta)
 - Si possono definire diversi tipi di comunicazione
 - ▶ Connection oriented (TCP) o connectionless (UDP)



Comunicazione Socket Unix

- Comunicazione Client-Server Unix
 - Server in ascolto

```
int fd1, fd2;  
struct sockaddr_in mio_indirizzo;
```

```
mio_indirizzo.sin_family    = AF_INET;  
mio_indirizzo.sin_port      = htons(5200);  
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
fd1 = socket(PF_INET, SOCK_STREAM, 0);  
bind(fd1, (struct sockaddr *) &mio_indirizzo, sizeof(mio_indirizzo));
```

```
listen(fd1, 5);  
fd2 = accept(fd1, NULL, NULL);  
...  
close(fd2);  
close(fd1);
```

Comunicazione Socket Unix

- Comunicazione Client-Server Unix
 - Client connesso

```
int fd;
struct sockaddr_in mio_indirizzo;

mio_indirizzo.sin_family    = AF_INET;
mio_indirizzo.sin_port      = htons(5200);
inet_aton("143.225.5.3", &indirizzo.sin_addr);

fd = socket(PF_INET, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *) &mio_indirizzo,
sizeof(mio_indirizzo));
...
close(fd);
```

Comunicazione Socket Unix

- Comunicazione Client-Server Unix
 - Schema di connessione

